

06_hyperparameter_analysis

February 16, 2026

1 06 — Hyperparameter analysis

Purpose: Analyze the relationship between hyperparameters and predictive multiplicity. Compute model-level variance contributions, correlate HP values with variance, and use the surrogate (HP importance) from the analysis module.

1.1 Imports and path setup

```
[13]: import sys
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from analysis.run_analysis import load_meta, load_P_test, load_split,
    ↪select_rashomon_global
from analysis.hyperparams import parse_hyperparameters,
    ↪variance_decomposition_by_hyperparameter, compute_hp_importance

RESULTS_DIR = ROOT / "results"
```

1.2 Load meta and P_test for one run; select Rashomon

Load one run (e.g. first seed), select top-K by validation Brier, and get predictions P and meta for Rashomon models only.

```
[14]: DATASET = "compas"
K = 25

dataset_dir = RESULTS_DIR / DATASET
```

```

run_dirs = sorted(dataset_dir.glob("seed=*"), key=lambda p: int(p.name.
↳split("=")[1]) if "=" in p.name else 0)
if not run_dirs:
    raise SystemExit(f"No run dirs in {dataset_dir}")

run_dir = run_dirs[0]
meta = load_meta(run_dir)
P_test = load_P_test(run_dir)
n_cand = len(meta)
K_actual = min(K, n_cand)
idx = select_rashomon_global(run_dir, K=K_actual)

P_sel = P_test[idx]
meta_sel = meta.iloc[idx].reset_index(drop=True)
print(f"Run: {run_dir.name}, Rashomon size: {len(idx)}")
meta_sel.head()

```

Run: seed=0, Rashomon size: 25

```

[14]:  model_name  candidate_id  candidate_seed  \
0         GBM             30             30
1         GBM             40             40
2         GBM             23             23
3         GBM             31             31
4         GBM             29             29

```

		hp	val_brier	outer_seed
0	{"subsample": 1.0, "n_estimators": 200, "max_d...	0.206094	0	
1	{"subsample": 1.0, "n_estimators": 100, "max_d...	0.206263	0	
2	{"subsample": 1.0, "n_estimators": 500, "max_d...	0.206368	0	
3	{"subsample": 1.0, "n_estimators": 500, "max_d...	0.206467	0	
4	{"subsample": 1.0, "n_estimators": 100, "max_d...	0.206474	0	

1.3 Compute model-level variance scores

For each model m : $V_m = (1/n_{\text{test}}) * \sum_i (f_m(x_i) - \bar{f}(x_i))^2$ (contribution to total variance / distance to ensemble mean).

```

[15]: f_bar = np.mean(P_sel, axis=0)
V_m = np.mean((P_sel - f_bar) ** 2, axis=1)
meta_sel["model_variance"] = V_m
print("Model-level variance (mean):", V_m.mean())
meta_sel[["model_name", "val_brier", "model_variance"]].head(10)

```

Model-level variance (mean): 0.0008409453700273324

```

[15]:  model_name  val_brier  model_variance
0         GBM    0.206094         0.000286

```

1	GBM	0.206263	0.000471
2	GBM	0.206368	0.000682
3	GBM	0.206467	0.000418
4	GBM	0.206474	0.000366
5	GBM	0.206565	0.000473
6	GBM	0.206609	0.000191
7	GBM	0.206669	0.000480
8	GBM	0.206707	0.000497
9	GBM	0.206835	0.000881

1.4 Boxplot: variance by model family

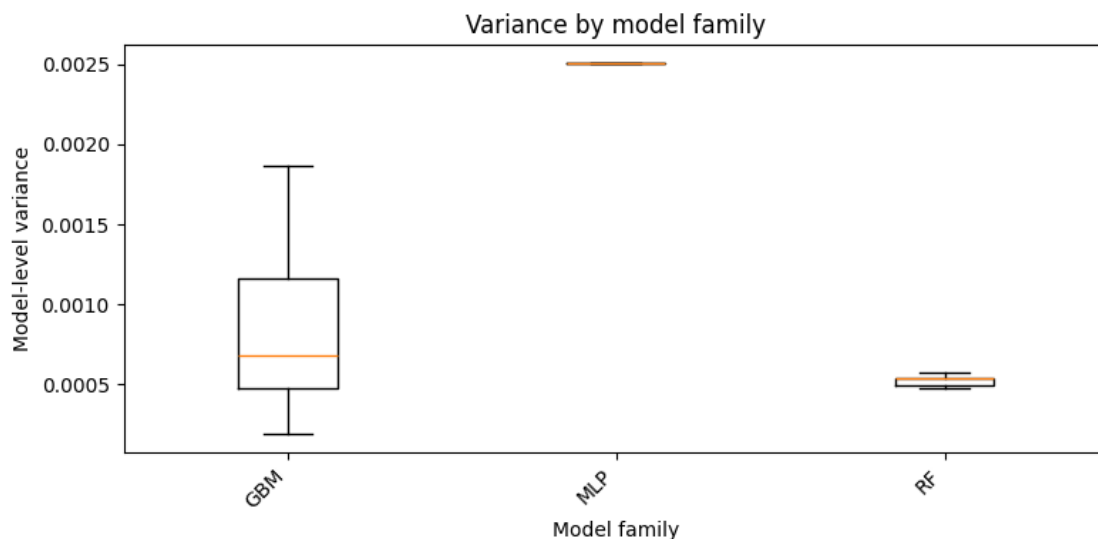
Distribution of model-level variance (contribution to multiplicity) for each model family in the Rashomon set.

```
[16]: families = sorted(meta_sel["model_name"].unique())
data_by_family = [meta_sel.loc[meta_sel["model_name"] == f, "model_variance"].
    values for f in families]
fig, ax = plt.subplots(figsize=(8, 4))
ax.boxplot(data_by_family, labels=families)
ax.set_xlabel("Model family")
ax.set_ylabel("Model-level variance")
ax.set_title("Variance by model family")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\2779024400.py:4:

MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_by_family, labels=families)
```



1.5 Parse hyperparameters and correlation analysis

Expand meta to hp_* columns and compute Spearman correlation between numeric HPs and model_variance.

```
[17]: meta_expanded = parse_hyperparameters(meta_sel)
      hp_cols = [c for c in meta_expanded.columns if c.startswith("hp_")]

      correlations = []
      for col in hp_cols:
          vals = meta_expanded[col].dropna()
          if vals.dtype in (np.float64, np.int64) or np.issubdtype(vals.dtype, np.
          ↪number):
              r = meta_expanded[col].corr(meta_expanded["model_variance"],
          ↪method="spearman")
              correlations.append({"hyperparameter": col.replace("hp_", ""),
          ↪"spearman": r})

      corr_df = pd.DataFrame(correlations).sort_values("spearman", key=abs,
          ↪ascending=False)
      print("Spearman correlation (HP vs model_variance):")
      display(corr_df)
```

Spearman correlation (HP vs model_variance):

```
c:\Users\dejvi\Documents\pythonProject\Rashomon Sets\rashomon-
multiplicity\.venv\Lib\site-packages\pandas\core\nanops.py:1632:
ConstantInputWarning: An input array is constant; the correlation coefficient is
not defined.
```

```
    return spearmanr(a, b)[0]

   hyperparameter  spearman
5  min_samples_split  0.632456
3    learning_rate  0.438390
6  min_samples_leaf  0.288675
0      subsample -0.212499
2      max_depth -0.178839
1    n_estimators -0.172756
4           n_jobs      NaN
7    max_features      NaN
8  n_iter_no_change      NaN
9         max_iter      NaN
10 learning_rate_init      NaN
11          alpha      NaN
```

1.6 Partial Dependence Plots (HP vs model variance)

For each hyperparameter: **numeric** HPs show scatter + binned mean of model-level variance; **categorical** HPs show boxplot of variance by level. Shows how each HP is associated with contribution to multiplicity.

```
[18]: # Ensure model_variance is available (meta_expanded comes from meta_sel which
      ↪has it)
df_pdp = meta_expanded.copy()
if "model_variance" not in df_pdp.columns:
    df_pdp["model_variance"] = meta_sel["model_variance"].values

n_hp = len(hp_cols)
n_cols = 4
n_rows = max(1, (n_hp + n_cols - 1) // n_cols)
fig, axes = plt.subplots(n_rows, n_cols, figsize=(4 * n_cols, 3 * n_rows))
axes = np.atleast_2d(axes)

for i, col in enumerate(hp_cols):
    ax = axes.flat[i]
    hp_name = col.replace("hp_", "")
    vals = df_pdp[col]
    var = df_pdp["model_variance"]
    valid = vals.notna()
    vals, var = vals[valid], var[valid]
    if len(vals) == 0:
        ax.text(0.5, 0.5, "No data", ha="center", va="center", transform=ax.
        ↪transAxes)
        ax.set_title(hp_name)
        continue
    is_numeric = np.issubdtype(vals.dtype, np.number) or vals.dtype in (np.
    ↪float64, np.int64)
    if is_numeric:
        ax.scatter(vals, var, alpha=0.7, s=40)
        try:
            df_bin = pd.DataFrame({"hp": vals, "var": var})
            n_bins = min(10, max(2, len(df_bin) // 2))
            df_bin["bin"] = pd.qcut(df_bin["hp"], q=n_bins, duplicates="drop")
            means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
            ax.plot(means["hp"], means["var"], color="red", linewidth=2,
            ↪label="mean")
        except Exception:
            pass
        ax.set_xlabel(hp_name)
    else:
        v_str = vals.astype(str)
        data_by_cat = [var[v_str == c].values for c in v_str.unique()]
        ax.boxplot(data_by_cat, labels=v_str.unique())
```

```

        ax.set_xlabel(hp_name)
        ax.tick_params(axis="x", rotation=45)
        ax.set_ylabel("Model variance")
        ax.set_title(hp_name)

for j in range(i + 1, axes.size):
    axes.flat[j].set_visible(False)
plt.suptitle("Partial dependence: HP value vs model-level variance")
plt.tight_layout()
plt.show()

```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:38:

MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been

renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_by_cat, labels=v_str.unique())
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:38:

MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_by_cat, labels=v_str.unique())
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:38:

MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_by_cat, labels=v_str.unique())
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:38:

MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_by_cat, labels=v_str.unique())
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:30: FutureWarning:

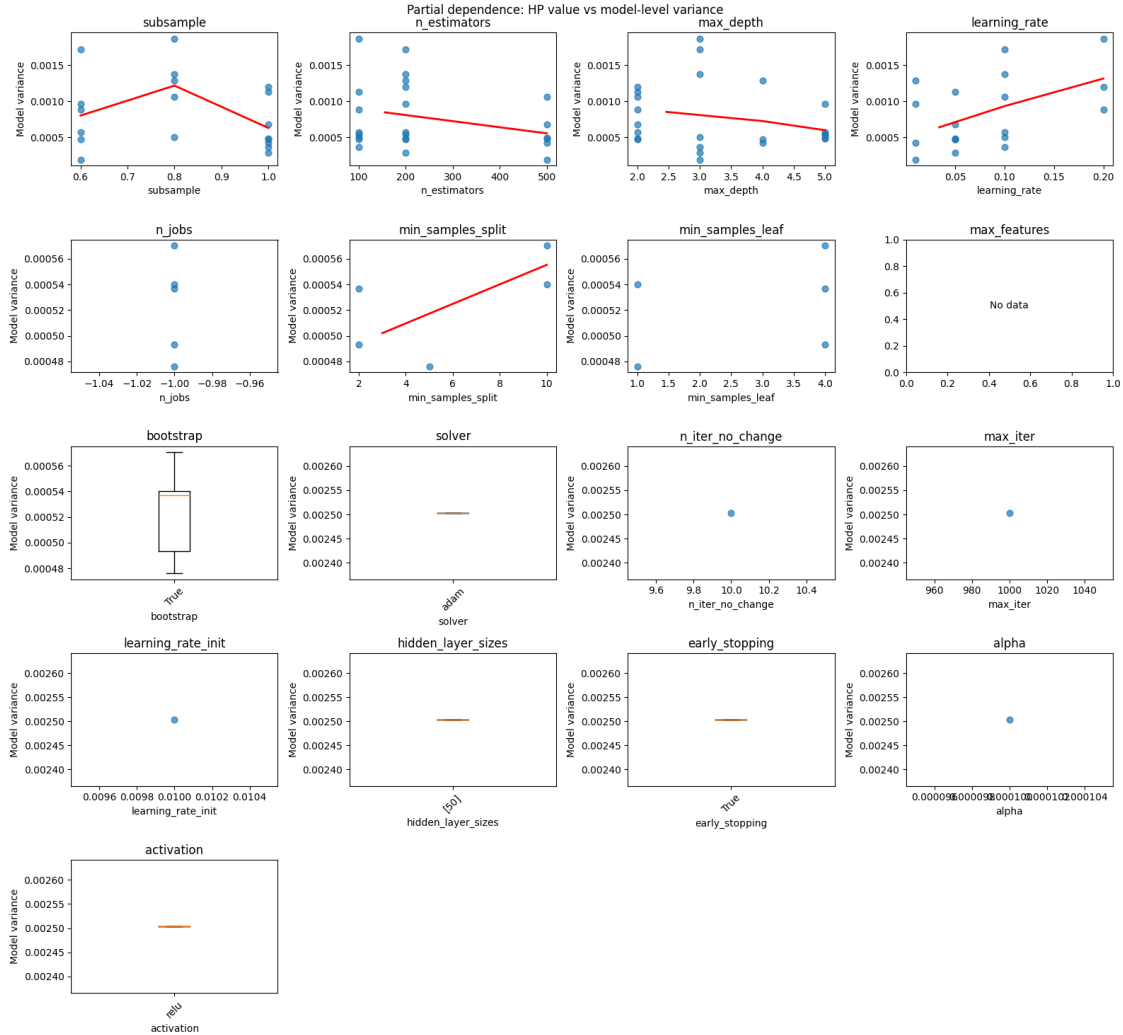
The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
means = df_bin.groupby("bin").agg({"hp": "mean", "var": "mean"})
```

C:\Users\dejvi\AppData\Local\Temp\ipykernel_6988\31700246.py:38:

MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(data_by_cat, labels=v_str.unique())
```



1.7 Surrogate model importance (Var_between / Var_total)

Use `analysis.hyperparams.compute_hp_importance` to rank hyperparameters by variance-between ratio.

```
[19]: importance_df = compute_hp_importance(P_sel, meta_expanded)
print("HP importance (mean Var_between / Var_total):")
display(importance_df)

if len(importance_df) > 0:
    fig, ax = plt.subplots(figsize=(8, max(4, len(importance_df) * 0.3)))
    ax.barh(importance_df["hyperparameter"],
            importance_df["mean_var_between_ratio"], color="steelblue")
    ax.set_xlabel("Mean Var_between / Var_total")
    ax.set_title("Hyperparameter importance (variance contribution)")
```

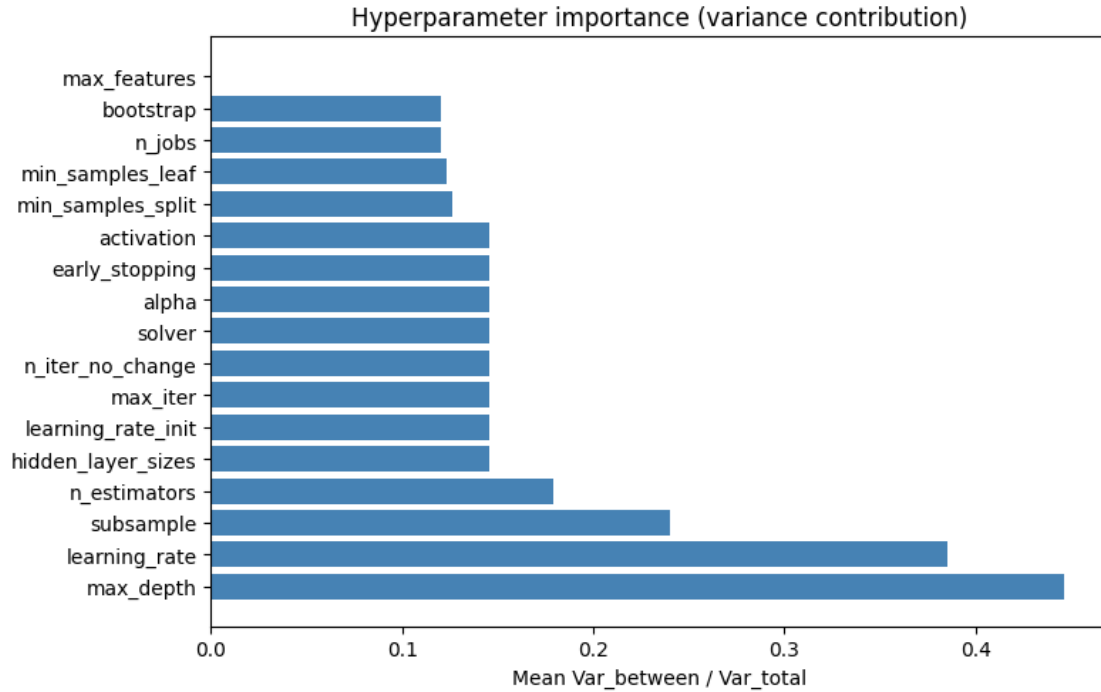


```
plt.tight_layout()
plt.show()
```

HP importance (mean Var_between / Var_total):

	hyperparameter	mean_var_between_ratio	mean_var_within_ratio \
2	max_depth	0.446310	0.553690
3	learning_rate	0.385055	0.614945
0	subsample	0.240297	0.759703
1	n_estimators	0.178950	0.821050
13	hidden_layer_sizes	0.145586	0.854414
12	learning_rate_init	0.145586	0.854414
11	max_iter	0.145586	0.854414
10	n_iter_no_change	0.145586	0.854414
9	solver	0.145586	0.854414
15	alpha	0.145586	0.854414
14	early_stopping	0.145586	0.854414
16	activation	0.145586	0.854414
5	min_samples_split	0.126310	0.873690
6	min_samples_leaf	0.123344	0.876656
4	n_jobs	0.120304	0.879696
8	bootstrap	0.120304	0.879696
7	max_features	0.000000	1.000000

	n_values	n_models
2	5	25
3	5	25
0	4	25
1	4	25
13	2	25
12	2	25
11	2	25
10	2	25
9	2	25
15	2	25
14	2	25
16	2	25
5	4	25
6	3	25
4	2	25
8	2	25
7	1	25



1.8 Interpretation

- **Model-level variance:** Higher V_m means model m deviates more from the ensemble mean; such models contribute more to predictive multiplicity.
- **Spearman correlation:** Identifies which HPs (e.g. depth, learning_rate) are associated with higher variance.
- **Var_between ratio:** HPs with high between-group variance explain more of the total prediction variance across the Rashomon set.